

Supervised Learning II: Classification Models

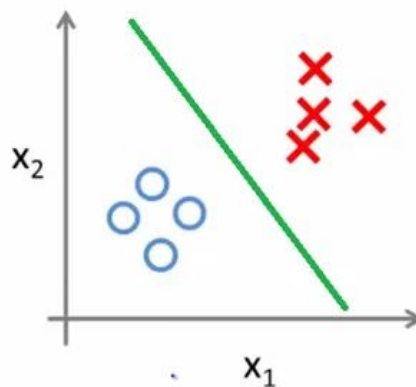
Instructor: Caryn Geady, PhD

Date: 2nd Feb 2026

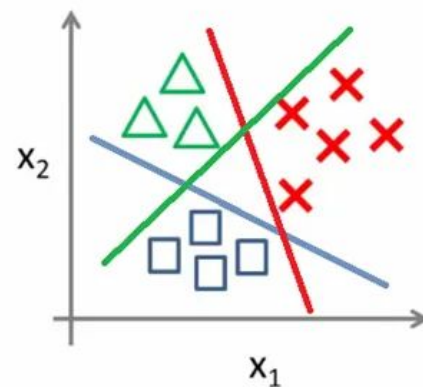
What is Classification?

- A **supervised learning task** where the goal is to predict **discrete labels** (categories) based on input features.
- Output:
 - **Binary** Classification (e.g., yes/no, true/false);
 - **Multiclass** Classification (e.g., categorizing images into multiple classes like car, bike, bus).

Binary classification:

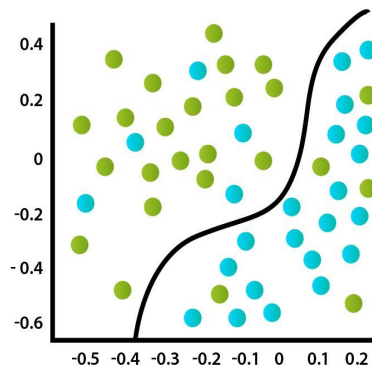


Multi-class classification:

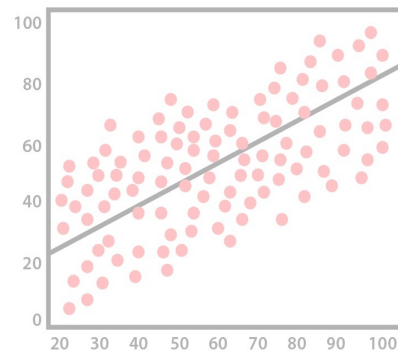


Today's Objectives

- Introduction to Classification Models:
 - k-Nearest Neighbours;
 - Decision Trees;
 - Random Forest;
 - Support Vector Machines;
- Use Cases;
- Metrics for Evaluation:
 - Key Metrics: Accuracy, Precision, Recall, F1 Score, AUROC, AUPRC;
- Regularization:
 - Pruning (for Decision Trees);
 - Hyperparameter tuning



CLASSIFICATION



REGRESSION

Classification Algorithms

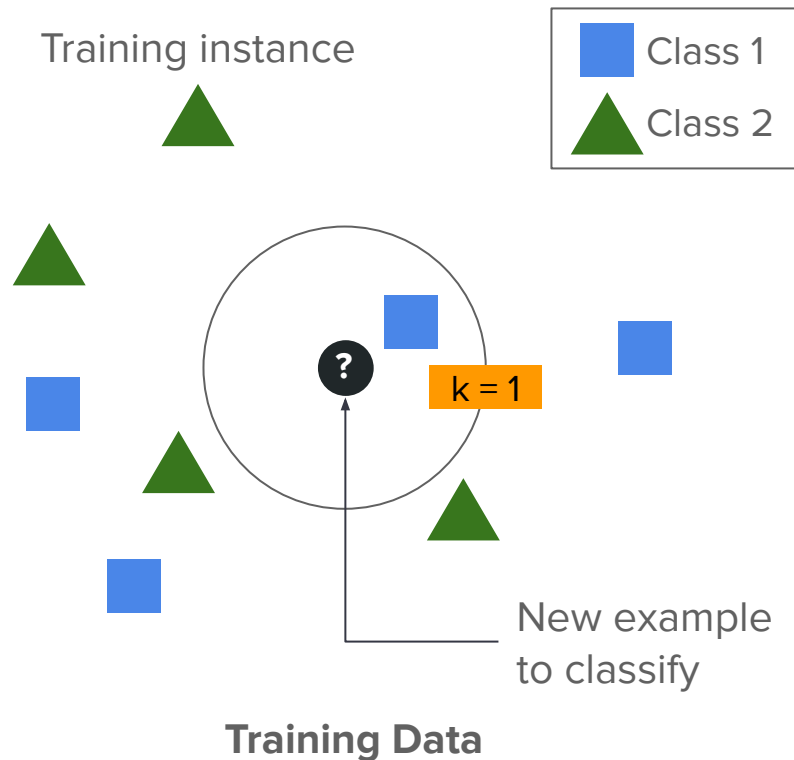
k-Nearest-Neighbors

- **Instance-based algorithm** that classifies a data point based on the majority label of its nearest neighbours;



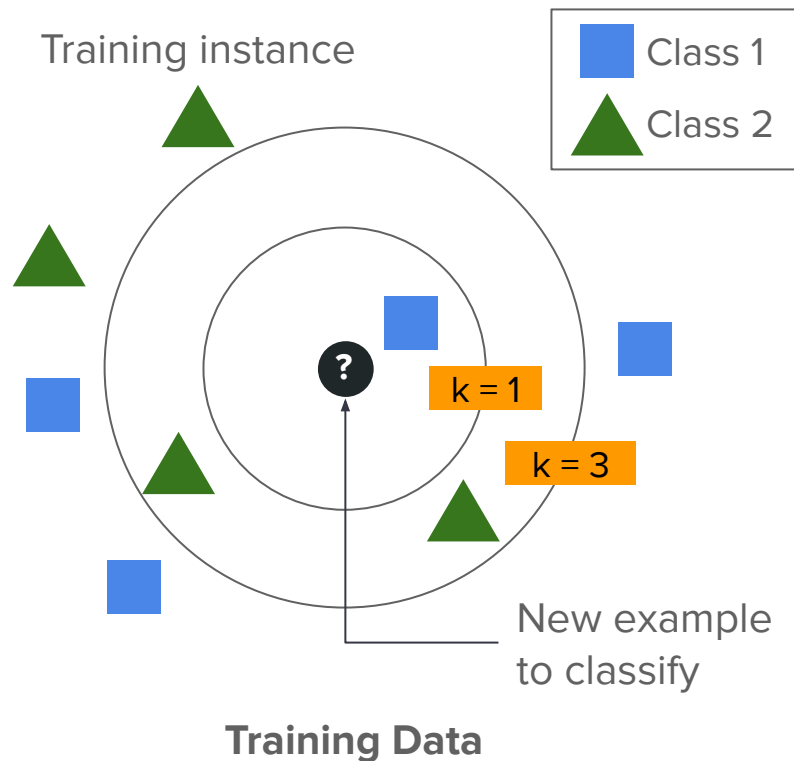
k-Nearest-Neighbors

- **Instance-based algorithm** that classifies a data point based on the majority label of its nearest neighbours;
- **Key concepts:**
 - **Number of neighbours (k)** → determines the size of the neighbourhood;



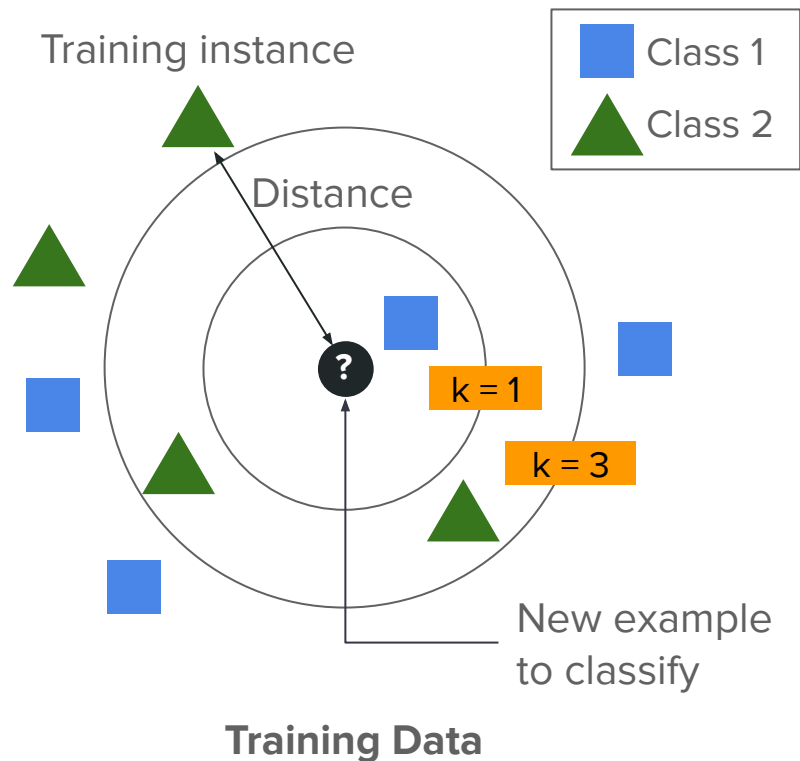
k-Nearest-Neighbors

- **Instance-based algorithm** that classifies a data point based on the majority label of its nearest neighbours;
- **Key concepts:**
 - **Number of neighbours (k)** → determines the size of the neighbourhood;



k-Nearest-Neighbors

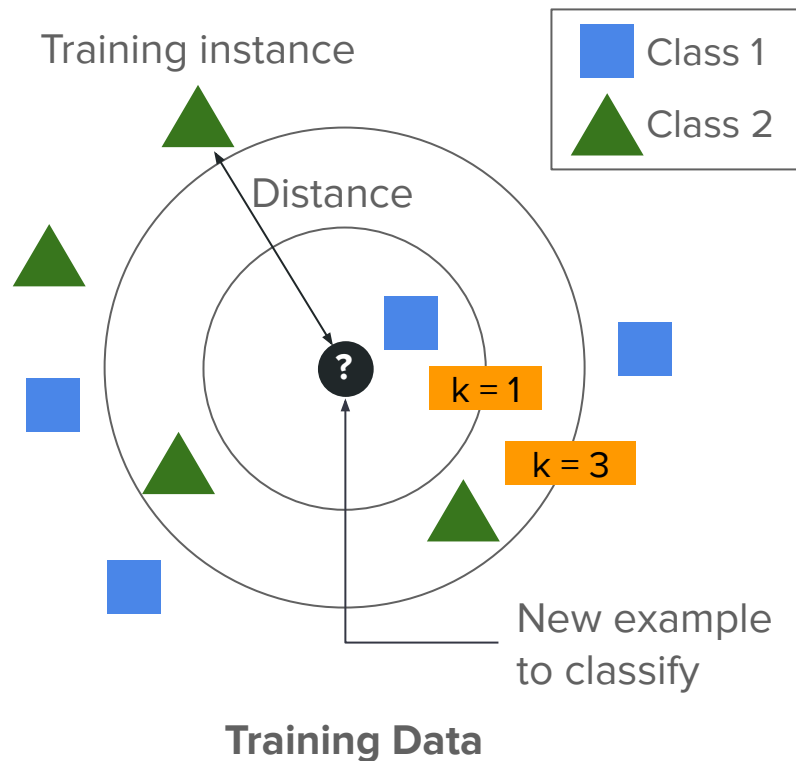
- **Instance-based algorithm** that classifies a data point based on the majority label of its nearest neighbours;
- **Key concepts:**
 - **Number of neighbours (k)** → determines the size of the neighbourhood;
 - **Distance metrics** → Euclidean, Manhattan, and more!



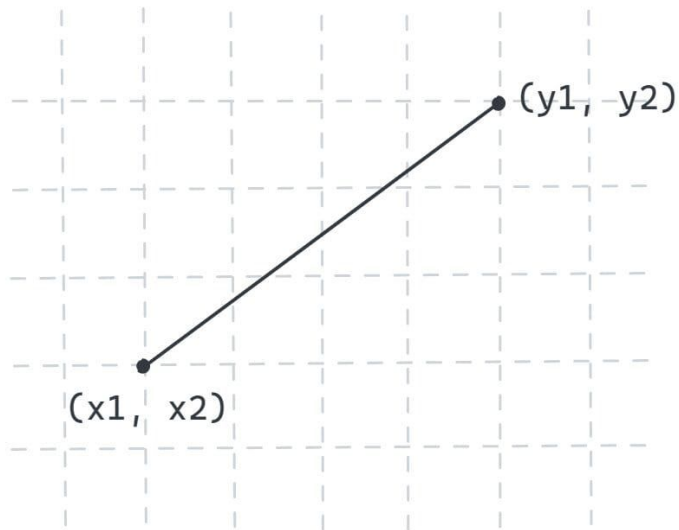


k-Nearest-Neighbors

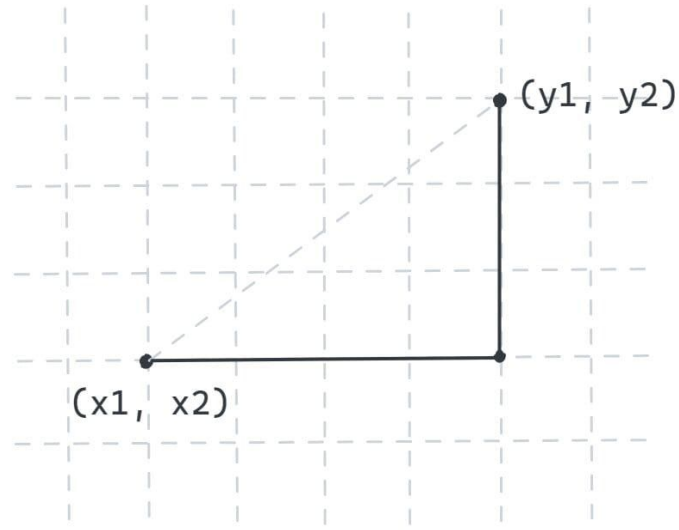
- **Instance-based algorithm** that classifies a data point based on the majority label of its nearest neighbours;
- **Key concepts:**
 - **Number of neighbours (k)** → determines the size of the neighbourhood;
 - **Distance metrics** → Euclidean, Manhattan, and more!
 - **Computational challenges** → performance slows for large datasets or high-dimensional data



k-Nearest-Neighbors - Distance Metrics



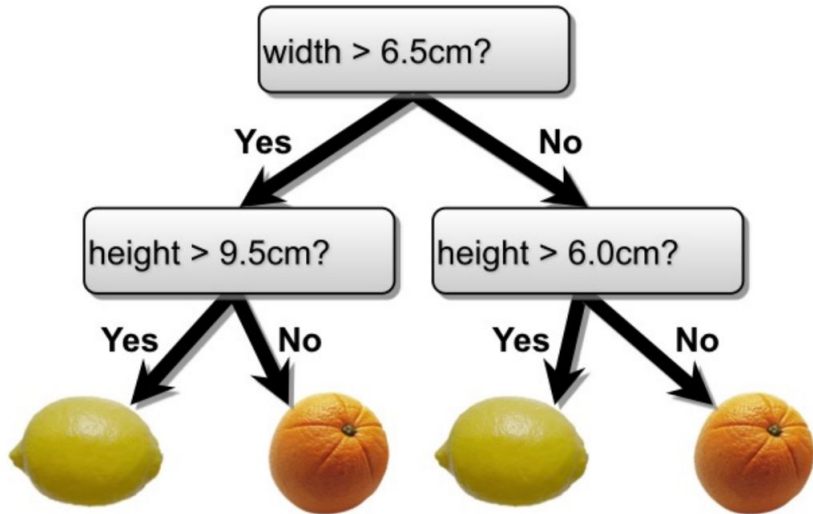
Euclidean Distance



Manhattan Distance

Decision Trees

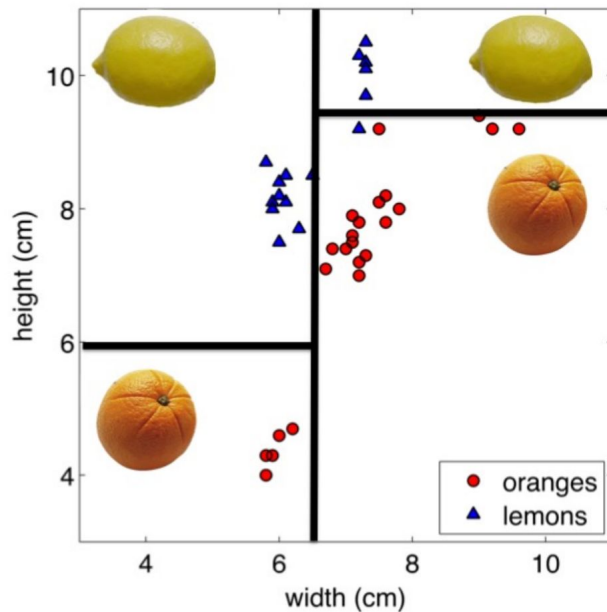
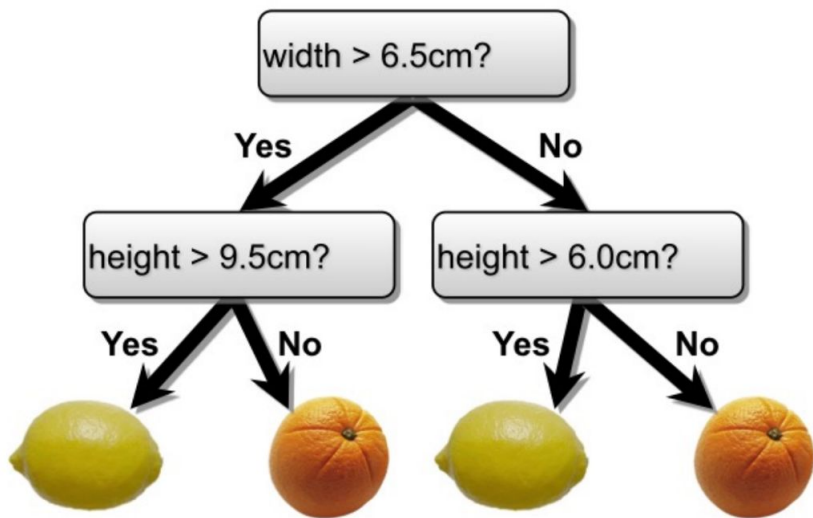
- Simple but powerful ML algorithm;
- A tree-like structure used for decision making and predictive modeling;



Decision Trees

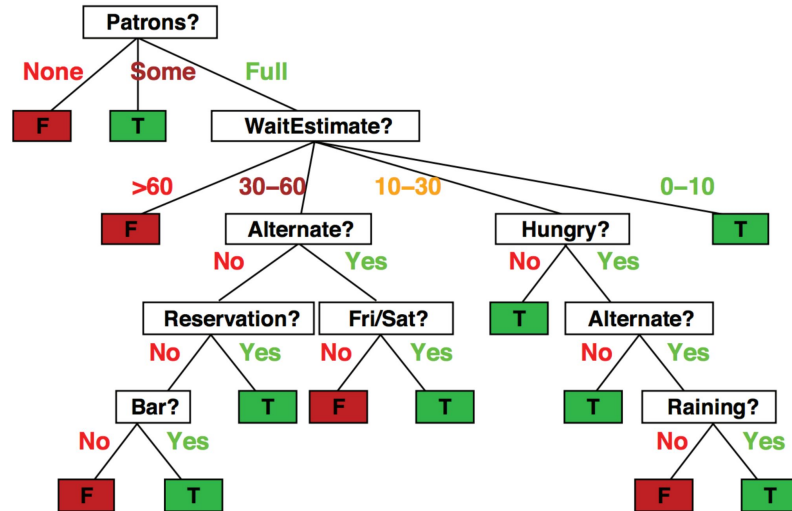


- Simple but powerful ML algorithm;
- A tree-like structure used for decision making and predictive modeling;



Decision Trees

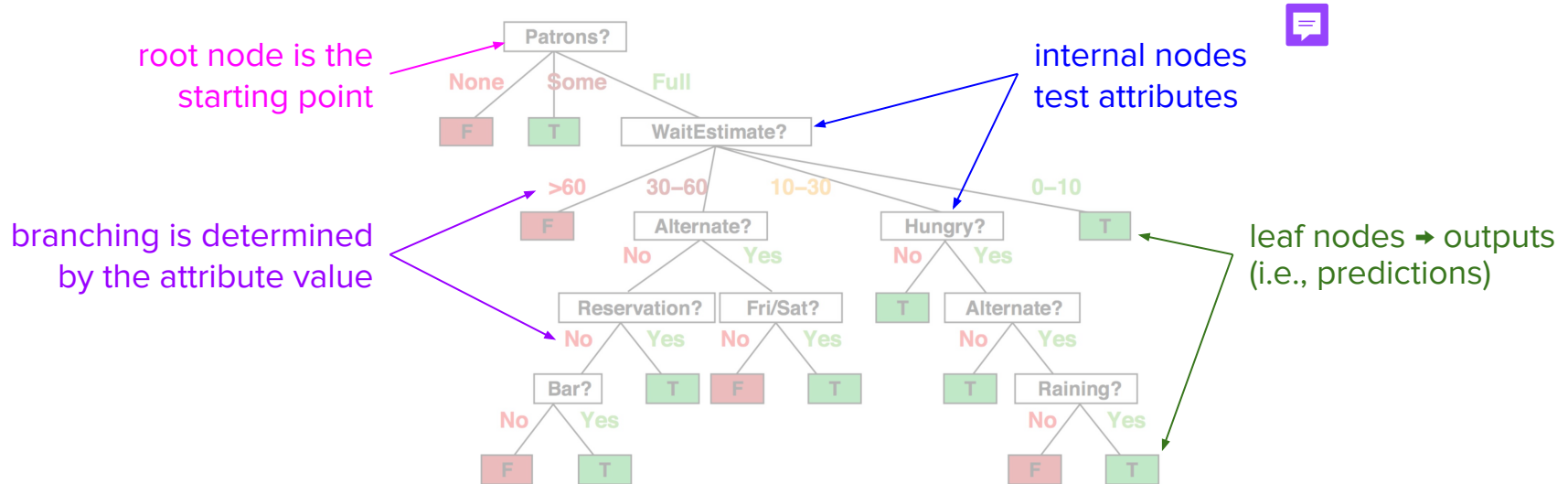
- Simple but powerful ML algorithm;
- A tree-like structure used for decision making and predictive modeling;
- Can be used with both continuous and discrete attributes (i.e., features);





Decision Trees

- Simple but powerful ML algorithm;
- A tree-like structure used for decision making and predictive modeling;
- Can be used with both continuous and discrete attributes (i.e., features);



Decision Trees - Splitting Criteria

Classification

Definition	Formula	Range	Key Characteristics
Gini Impurity 			
Likelihood of incorrect classification. Measures impurity of a split.	$\text{Gini} = 1 - \sum_i p_i^2$	0 (pure) $\rightarrow$ 0.5	Simple to calculate, less sensitive to small probability changes compared to entropy.
Entropy			
Measures uncertainty or disorder in a node.	$\text{Entropy} = - \sum_i p_i \log(p_i)$	0 (pure) $\rightarrow$ $\log(n)$	Sensitive to data distribution, penalizes both small and large probabilities.

Decision Trees - Splitting Criteria

Regression

Definition	Formula	Range	Key Characteristics
Mean Squared Error (MSE)			
Average squared difference between actual and predicted values.	$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y})^2$	$0 \rightarrow \infty$	Penalizes larger errors heavily, suitable for continuous target variables.

Decision Trees - Splitting Criteria

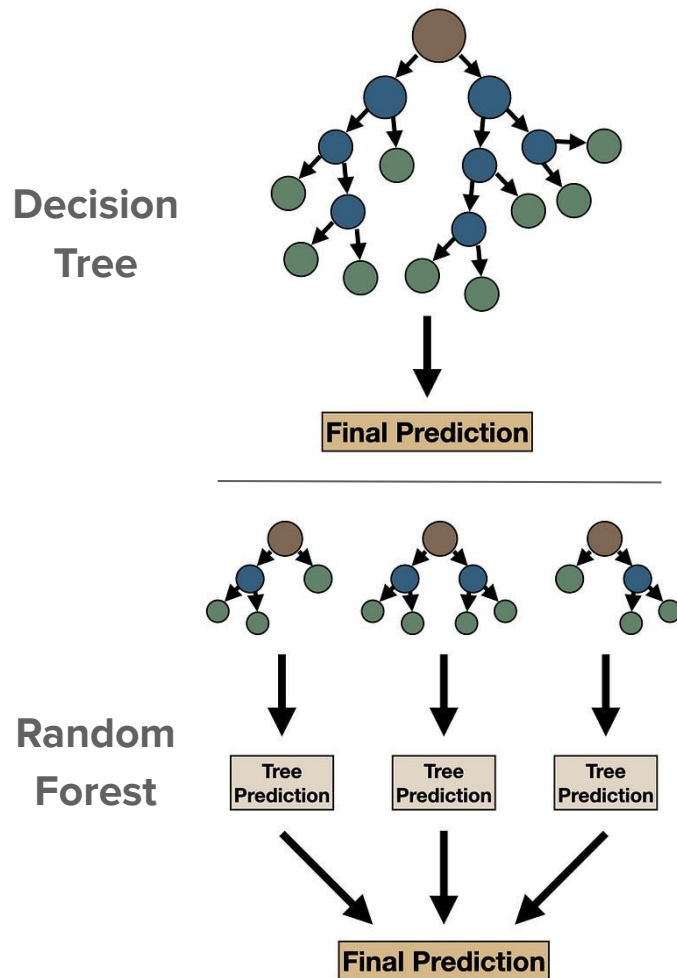
Both

Definition	Formula	Range	Key Characteristics
Information Gain			
Reduction in impurity after a split (Classification: Gini or Entropy; Regression: MSE).	$IG = \text{Impurity}_{\text{parent}} - \text{Weighted Impurity}_{\text{children}}$	Dependent on impurity metric	Drives the decision-making in splitting; higher gain indicates a better split.

Random Forest

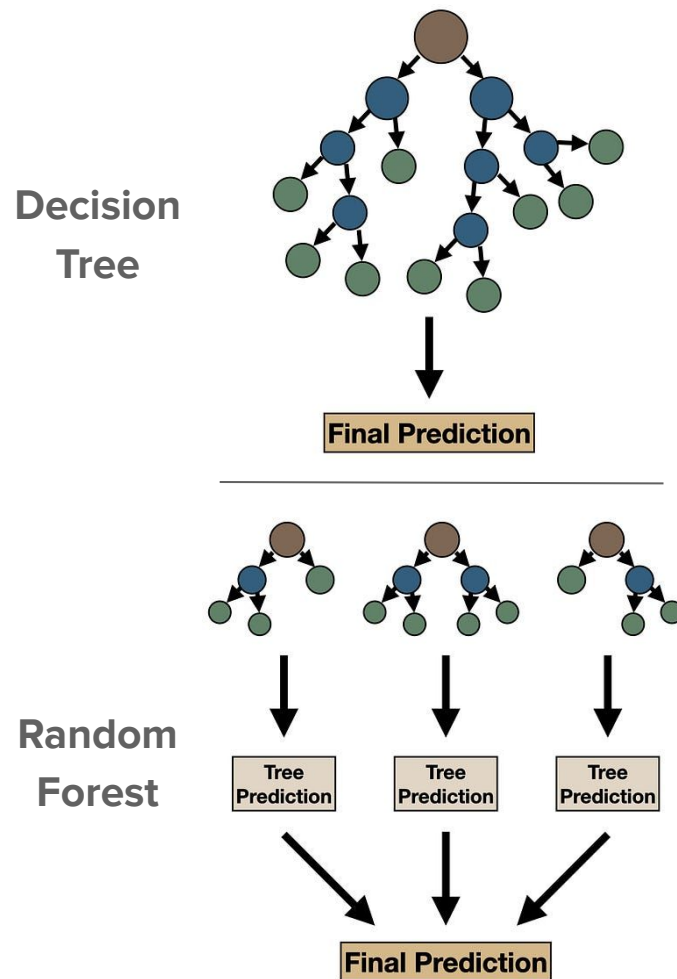


- **Ensemble learning method** combining multiple decision trees;
- Key concepts:
 - **Bagging** → randomly sampling data to create diverse trees; 
 - **Feature importance** → contribution of each feature to the prediction;
 - **Reduced overfitting** → less prone compared to individual decision trees



Training Random Forests

- **Bagging Method:**
 - **Bootstrap Sampling:** Randomly samples rows and features to create unique datasets for each tree.
 - **Aggregation:** Combines the outputs of all trees to reduce variance.
- **Reduces Overfitting:** Handles high-variance models (like decision trees) by smoothing predictions and focusing on signal over noise.



Seeing the Forest Through the Trees?

Feature	Decision Tree	Random Forest
Structure	Single tree model.	Combination of multiple decision trees.
Overfitting	Higher likelihood.	Reduced due to ensemble approach.
Accuracy	Lower accuracy.	More accurate and stable.
Computation	Faster, low computation.	Slower, high computation.
Visualization	Simple and interpretable.	Complex, harder to interpret.
Ideal For	Small datasets with straightforward relationships; quick insights and visualizations.	Large datasets with complex relationships; requires careful tuning (e.g., number of trees)

Support Vector Machines

- Finds a hyperplane separating classes with the **maximum margin**.
- **Support Vectors**: Data points defining the margin and decision boundary.

Types of SVMs

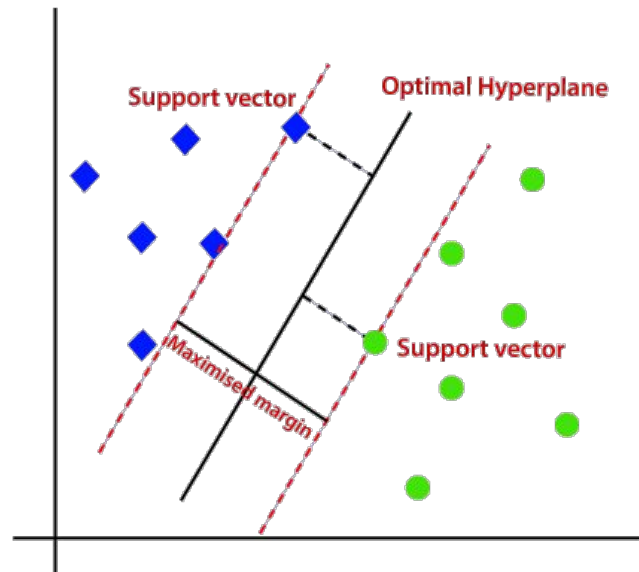
1. Linear SVMs:



- Uses **hard-margin** (no misclassification) or **soft-margin** (allows slack for misclassified points).
- **Hyperparameter C** adjusts margin flexibility.

2. Nonlinear SVMs:

- Handles non-linearly separable data with **kernel trick** to map data to higher dimensions.
- Common kernels: **Polynomial**, **RBF**, **Sigmoid**.



Use Cases

Classification Use Cases

- **General:**

- Spam email detection;
- Fraud detection;
- Image recognition → dog versus cat;

- **Healthcare:**

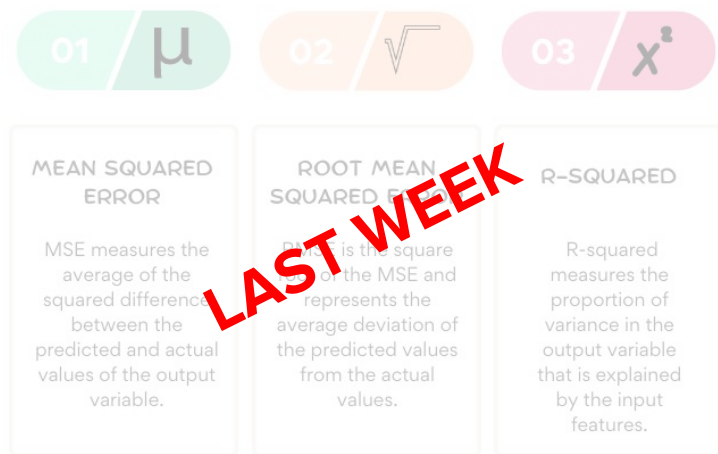
- Medical diagnosis → malignant versus benign;
- Rare disease detection;
- Risk stratification → high versus low risk patients;
- Treatment response prediction



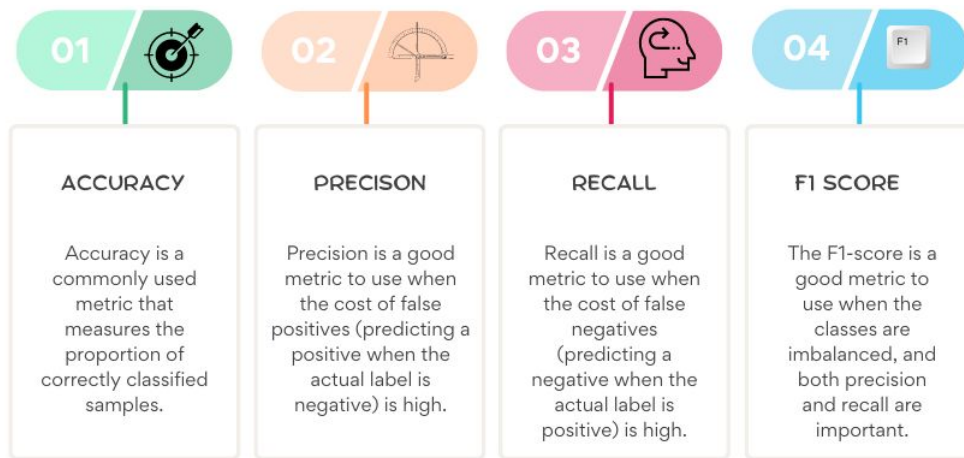
Metrics for Evaluation

Evaluating Classification Models

- Objective evaluation of model performance → metric selection is crucial!
- Some examples:



Regression



Classification

Evaluation Metrics for Classification

	Positive Prediction	Negative Prediction
Positive Class	True Positive (TP)	False Negative (FN)
Negative Class	False Positive (FP)	True Negative (TN)

Confusion Matrix

Evaluation Metrics for Classification



	Positive Prediction	Negative Prediction
Positive Class	True Positive (TP)	False Negative (FN)
Negative Class	False Positive (FP)	True Negative (TN)

Confusion Matrix

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{F1}_{\text{score}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$$

Evaluation Metrics for Classification

	Positive Prediction	Negative Prediction
Positive Class	True Positive (TP)	False Negative (FN)
Negative Class	False Positive (FP)	True Negative (TN)

Confusion Matrix

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F1_{\text{score}} = \frac{2TP}{2TP + FP + FN}$$

Evaluation Metrics for Classification

	Positive Prediction	Negative Prediction
Positive Class	True Positive (TP)	False Negative (FN)
Negative Class	False Positive (FP)	True Negative (TN)

Confusion Matrix

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$



$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F1_{\text{score}} = \frac{2TP}{2TP + FP + FN}$$

Evaluation Metrics for Classification

	Positive Prediction	Negative Prediction
Positive Class	True Positive (TP)	False Negative (FN)
Negative Class	False Positive (FP)	True Negative (TN)

Confusion Matrix

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F1_{\text{score}} = \frac{2TP}{2TP + FP + FN}$$

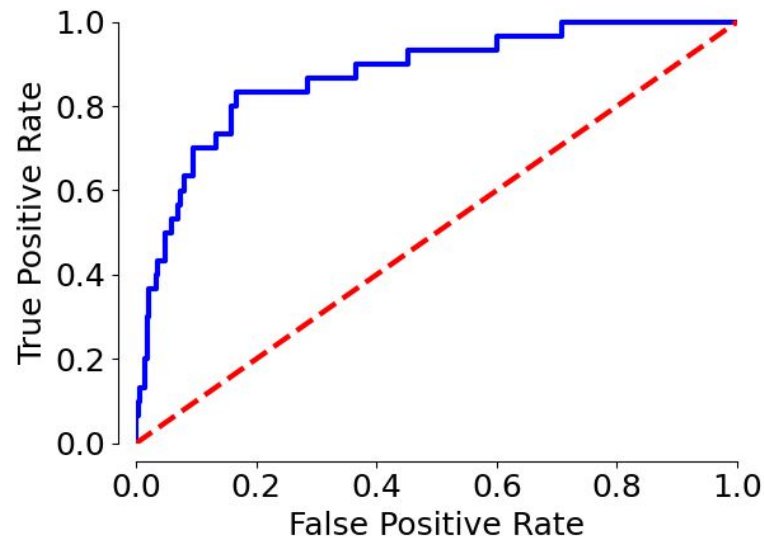
Evaluation Metrics for Classification

$$TPR = \frac{TP}{TP + FN}$$

	Positive Prediction	Negative Prediction
Positive Class	True Positive (TP)	False Negative (FN)
Negative Class	False Positive (FP)	True Negative (TN)

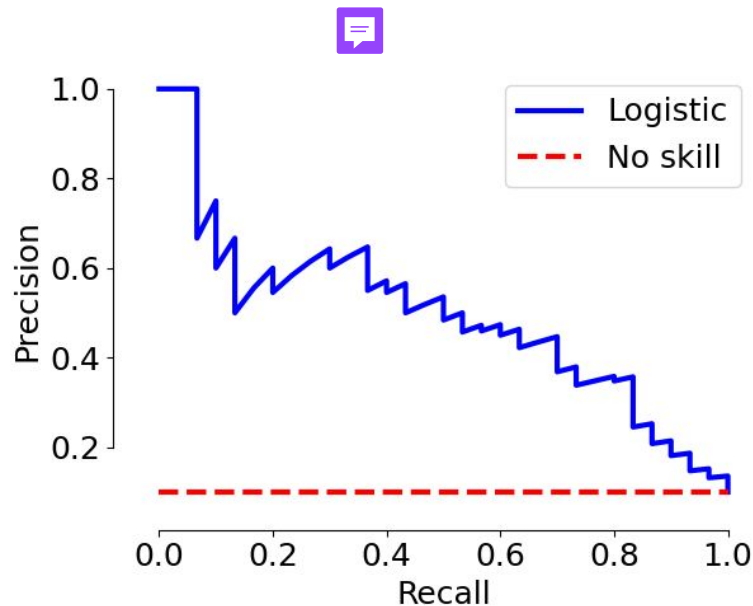
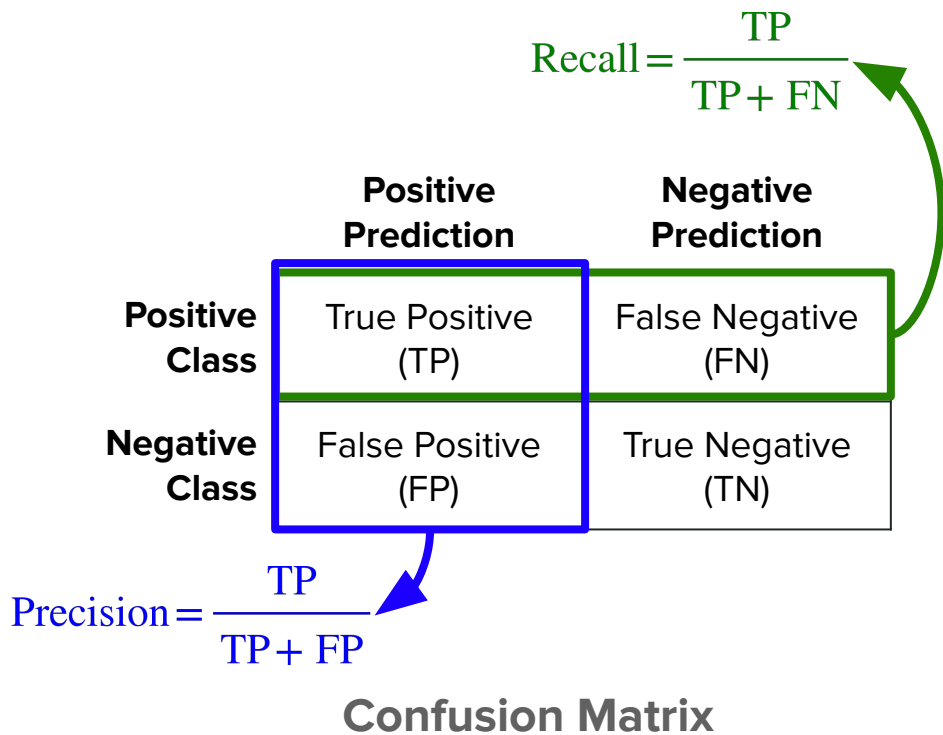
$$FPR = \frac{FP}{FP + TN}$$

Confusion Matrix

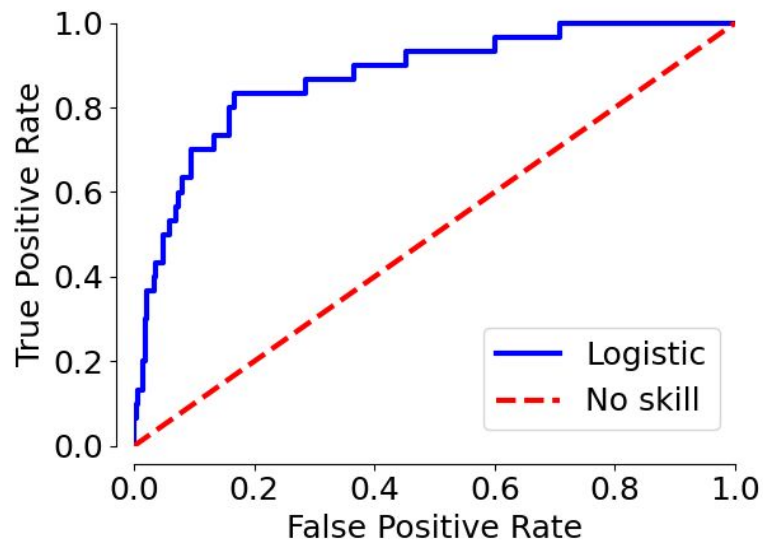


Receiver Operating Characteristic

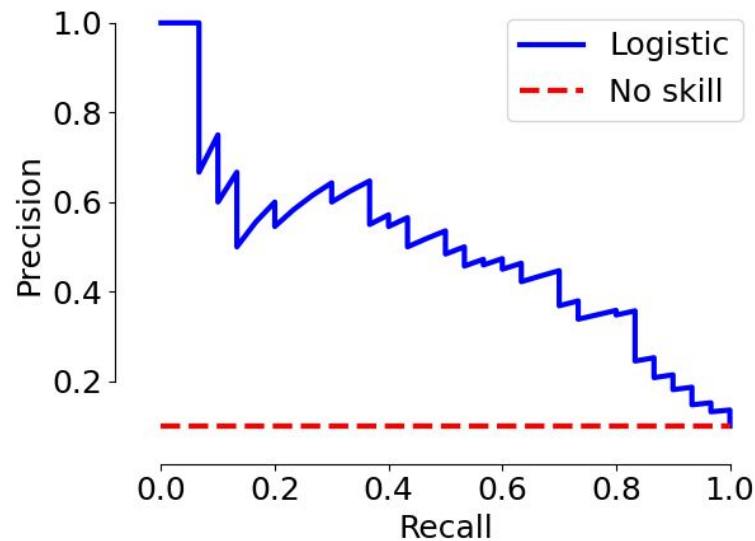
Evaluation Metrics for Classification



Evaluation Metrics for Classification



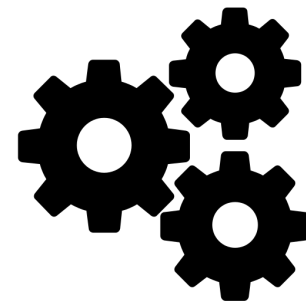
Receiver Operating Characteristic



Precision-Recall Curve

Regularization

Hyperparameters vs. Model Parameters



Aspect	Hyperparameters	Model Parameters
Definition	Set externally before training	Learned from data during training
Examples	Number of neighbors (k), number of trees, C-parameter	Weights, biases, split thresholds
Purpose	Control model structure or training process	Map inputs to outputs
Tuning	Adjusted manually or via optimization methods	Automatically optimized via training



Hyperparameter Tuning

- Process of experimenting with **different hyperparameter configurations** to optimize model performance.
- Involves **sequential training** of models with **varied hyperparameters**.
- **Controls model structure and performance.**
- **Balances overfitting and underfitting** for accurate predictions.
- **Techniques:**
 - **Grid Search:** Exhaustive search through combinations.
 - **Random Search:** Randomly samples hyperparameter space.
 - **Bayesian Optimization:** Uses probabilistic models to select optimal configurations.

Regularization in Classification

Algorithm	Regularization Techniques
K-Nearest Neighbors (KNN)	- Optimal K: Larger K reduces overfitting; smaller K captures local patterns. - Distance Metric: Choose appropriate metrics (Minkowski, Manhattan, Mahalanobis). - Feature Scaling: Standardize or normalize data. - Dimensionality Reduction: Use PCA or feature selection to reduce irrelevant features.
Decision Trees	- Maximum Depth (max_depth): Limit tree depth to avoid capturing noise. - Minimum Samples per Split (min_samples_split): Control the minimum samples needed to split. - Minimum Samples per Leaf (min_samples_leaf): Ensure sufficient samples in leaf nodes. - Maximum Features (max_features): Restrict features considered for splitting. - Pruning: Apply cost-complexity or post-pruning techniques.

Regularization in Classification

Algorithm	Regularization Techniques
Random Forest	- Number of Trees ($n_estimators$): Use more trees for stability. - Maximum Depth (max_depth): Limit tree depth for simpler models. - Minimum Samples per Split/Leaf: Same as decision trees. - Maximum Features ($max_features$): Limit features for splitting to increase decorrelation. - Bootstrap Sampling: Ensure proper row sampling (bagging).
Support Vector Machines (SVM)	- Regularization Parameter (C): Smaller C reduces overfitting by allowing more margin violations. - Kernel Choice: Use appropriate kernels (linear, polynomial, RBF). - Kernel-Specific Parameters: Tune degree (polynomial) or $gamma$ (RBF) to balance complexity. - Feature Scaling: Standardize features. - Dimensionality Reduction: Simplify feature set to improve generalization.

Summary and Q&A

Summary – Algorithms

Algorithm	Strengths	Weaknesses	Hyperparameters*
KNN	Simple, interpretable	Sensitive to noise, requires scaling	k (neighbors), distance metric
Decision Trees	Interpretable, fast to train	Prone to overfitting	Max depth, min samples per split
Random Forest	Reduces overfitting, robust	Can be slow with many trees	Number of trees, max features
SVM	Effective in high-dimensional spaces	Computationally expensive	Kernel type, C, gamma

* Key

Summary – Evaluation Metrics

Metric	Definition	Best Use Case
Accuracy	Overall correctness	Balanced datasets
Precision	Positive predictive value	When false positives matter
Recall	Sensitivity/True positive rate	When false negatives matter
F1 Score	Precision-recall balance	Imbalanced datasets
AUROC	Distinguishes between classes	Overall performance
AUPRC	Focus on precision-recall tradeoff	Imbalanced datasets

Questions?
